OS Orange Problem

Name: Chiranthan Sathrasala

SRN: PES1UG24CS565

Screenshots:

Phase1:

1a

```
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1b

```
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$
```

Phase2:

2a

```
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2b

```
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ xxd .pes/objects/2c/f8d83d9ee29543b34a
87727421fdecb7e3f3a183d337639025de576db9ebb4 | head -20
00000000: 626c 6f62 2036 0068 656c 6c6f 0a         blob 6.hello.
```

OS Orange Problem

Phase3:

3a

```
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ echo "hello" > file1.txt
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ echo "world" > file2.txt
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ ./pes add file1.txt file2.txt
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  commit.c
  untracked:  commit.h
  untracked:  index.c
  untracked:  index.h
  untracked:  Makefile
  untracked:  object.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  README.md
  untracked:  test_objects
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  test_tree
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  tree.h
```

3b

```
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ cat .pes/index
100755 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776418142 6 file1.txt
100755 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776418142 6 file2.txt
```

OS Orange Problem

Phase4:

4a

```
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ mkdir -p .pes/refs/heads .pes/objects
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ echo "hello" > file1.txt
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ ./pes add file1.txt
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ ./pes commit -m "Initial commit"
Committed: 4c99a6e69698... Initial commit
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ echo "Goodbye" > bye.txt
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ ./pes add bye.txt
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ ./pes commit -m "Add farewell"
Committed: b5df77e3679b... Add farewell
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ ./pes log
commit b5df77e3679ba1a22695b8bc53f9c8e7e0182802485b353285242c4b76a3996f
Author: Chianthan Sathrasala <PES1UG24CS565>
Date:   1776428644

    Add farewell

commit 4c99a6e69698bc81cf18604bf6690b62fe9cc00edd1c00c58ceb55ef288a09a5
Author: Chianthan Sathrasala <PES1UG24CS565>
Date:   1776428644

    Initial commit
```

4b

```
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/4c/99a6e69698bc81cf18604bf6690b62fe9cc00edd1c00c58ceb55ef288a09a5
.pes/objects/50/5ed45cdfddad22a20fc6f5af588a46932a9bd0164170a9e4220422c632faef
.pes/objects/b5/df77e3679ba1a22695b8bc53f9c8e7e0182802485b353285242c4b76a3996f
.pes/objects/be/61da8577c561282e5c72049f49d6e80fd0413e91f3ea914f33c16b8f80e73b
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

4c

```
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ cat .pes/refs/heads/main
b5df77e3679ba1a22695b8bc53f9c8e7e0182802485b353285242c4b76a3996f
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$
```

OS Orange Problem

Phase5:

Q5.1: A branch in Git is just a file... Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

Answer: To implement checkout, you must first update the .pes/HEAD file to point to the new branch's reference file (e.g., ref: refs/heads/<branch>). Next, you must update the .pes/index to reflect the snapshot of the new branch. Finally, the working directory must be updated by reading the blobs from the new branch's tree and overwriting the local files. The complexity stems from safely transitioning the working directory without causing data loss. You have to handle edge cases like files existing in the current branch but not the target branch (which must be deleted), handling nested directories dynamically, and ensuring that any uncommitted, unsaved work by the user isn't blindly overwritten by the new branch's files.

Q5.2: Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Answer: You can detect a dirty directory by doing a two-step comparison. First, iterate through the .pes/index. For each entry, use the stat() syscall to compare the actual file's mtime (modified time) and size against the metadata stored in the index. If they differ, the file is "modified." Second, if a file is modified, compare the hash of the file in the current index against the hash of that same file in the target branch's tree (inside the object store). If the hashes are different, it means the checkout process would overwrite the file to match the target branch. Since the file is also modified locally, overwriting it would destroy the user's uncommitted work, so the checkout must abort and refuse the operation.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly... What happens if you make commits in this state? How could a user recover those commits?

Answer: If you make commits in a Detached HEAD state, the .pes/HEAD file simply updates to contain the new commit's hash directly, but no branch reference file in refs/heads/ is updated. If the user subsequently checks out a different branch, those commits become "orphaned" because there is no branch name pointing to them to keep them reachable. To recover them, the user needs the exact 64-character hash of the last orphaned commit. They can then manually create a new file in .pes/refs/heads/ (a new branch) and paste that hash inside it. This reattaches the commit chain to a human-readable branch name, making it accessible again.

OS Orange Problem

Phase6:

Q6.1: Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently?

Answer: This requires a classic "Mark and Sweep" graph traversal algorithm.

1. **Mark Phase:** Start at the roots (every branch file in .pes/refs/heads/ and the HEAD file). Read the commit hashes. Do a Depth-First Search (DFS) following the parent pointers of each commit. For every commit visited, recursively traverse its tree and sub-trees to find all blob hashes.
2. **Tracking Data Structure:** As you visit each object, insert its hash into a **Hash Table** (or a Hash Set). A Hash Table is ideal here because insertions and lookups operate in $O(1)$ time complexity, which is crucial for millions of objects.
3. **Sweep Phase:** Iterate through the physical files in the .pes/objects/ shard directories. For each file, check if its hash exists in the Hash Table. If it does not, it is unreachable and can be safely deleted using unlink().

Estimation: For 100,000 commits, you must visit all 100,000 commit objects. Even with extensive deduplication of blobs, you would likely need to traverse and hash-check millions of object references to build the complete set of reachable nodes.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition... How does Git's real GC avoid this?

Answer: A race condition occurs if a user stages a file (pes add) exactly as the GC is running. The pes add command creates a new blob object in .pes/objects/. Because the user hasn't run pes commit yet, this new blob is not linked to any tree or branch. The concurrent GC sweep will view this brand-new blob as an "unreachable" orphan and delete it. Seconds later, when pes commit tries to link that blob into a tree, the repository becomes corrupted because the object is missing.

Git avoids this race condition by implementing a grace period based on file timestamps. Real Git GC only deletes unreferenced objects if their filesystem modification time (mtime) is older than a specific threshold (e.g., 2 weeks). This guarantees that freshly created, uncommitted objects are perfectly safe from the sweep.

OS Orange Problem

Final Screenshot:

Part1

```
seed@VM: ~/.../PES1UG24CS565-pes-vcs
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: c29800a5cae7... Initial commit
```

Part2

```
seed@VM: ~/.../PES1UG24CS565-pes-vcs
--- First Commit ---
Committed: c29800a5cae7... Initial commit

Log after first commit:
commit c29800a5cae7d751c9efec111ba37fe79b60893eec60a418f50537aec1275d9b
Author: Chianthan Sathrasala <PES1UG24CS565>
Date:   1776429751

    Initial commit


--- Second Commit ---
Committed: 1a15e2e1dbca... Update file.txt

--- Third Commit ---
Committed: 100fa1715503... Add farewell

--- Full History ---
commit 100fa1715503454adad06f035532225c33d63463805149677b714dc8d81c2cd6
Author: Chianthan Sathrasala <PES1UG24CS565>
Date:   1776429751

    Add farewell

commit 1a15e2e1dbca4894f12d5d5026f67cd93252f2a2a3297952d123d8e779664bc0
Author: Chianthan Sathrasala <PES1UG24CS565>
```

OS Orange Problem

Part3

```
seed@VM: ~/.../PES1UG24CS565-pes-vcs
commit 1a15e2e1dbca4894f12d5d5026f67cd93252f2a2a3297952d123d8e779664bc0
Author: Chianthan Sathrasala <PES1UG24CS565>
Date:   1776429751

    Update file.txt

commit c29800a5cae7d751c9efec111ba37fe79b60893eec60a418f50537aec1275d9b
Author: Chianthan Sathrasala <PES1UG24CS565>
Date:   1776429751

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
100fa1715503454adad06f035532225c33d63463805149677b714dc8d81c2cd6

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/0fa1715503454adad06f035532225c33d63463805149677b714dc8d81c2cd6
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/1a/15e2e1dbca4894f12d5d5026f67cd93252f2a2a3297952d123d8e779664bc0
```

Part4

```
seed@VM: ~/.../PES1UG24CS565-pes-vcs

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
100fa1715503454adad06f035532225c33d63463805149677b714dc8d81c2cd6

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/0fa1715503454adad06f035532225c33d63463805149677b714dc8d81c2cd6
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/1a/15e2e1dbca4894f12d5d5026f67cd93252f2a2a3297952d123d8e779664bc0
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/c2/9800a5cae7d751c9efec111ba37fe79b60893eec60a418f50537aec1275d9b
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
[04/17/26]seed@VM:~/.../PES1UG24CS565-pes-vcs$
```